DevOps Final Project - Winter 2026

Student: César Núñez
Date: 03/11/2026

Phase 1: Repository Setup and Version Control

Objective: Establish a professional version control foundation that supports collaborative development, automated deployments, and code quality standards.

Deliverables:

- Four properly structured Git repositories
- Documentation explaining your Git workflow and branching strategy
- Screenshots demonstrating:
    - Initial setup of main and develop branches
        - database:

```
canun:database (main)$ git branch --all
* main
  remotes/origin/main
canun:database (main)$ git switch -c develop
Switched to a new branch 'develop'
canun:database (develop)$ git branch --all
* develop
  main
  remotes/origin/main
```

        - frontend-service:

```
canun:frontend-service (main)$ git branch -r
  origin/main
canun:frontend-service (main)$ git branch -r --all
* main
  remotes/origin/main
canun:frontend-service (main)$ git switch -c dev
Switched to a new branch 'dev'
canun:frontend-service (dev)$ git branch --all
* dev
  main
  remotes/origin/main
```

        - order-service:

```
canun:order-service (main)$ git branch --al
* main
  remotes/origin/main
canun:order-service (main)$ git switch -c dev
Switched to a new branch 'dev'
canun:order-service (dev)$ git branch --all
* dev
  main
  remotes/origin/main
```

- product-service:

```
canun:product-service (main)$ git branch --all
* main
  remotes/origin/main
canun:product-service (main)$ git switch -c dev
Switched to a new branch 'dev'
canun:product-service (dev)$ git branch --all
* dev
  main
  remotes/origin/main
```

- Complete feature development workflow (from branch creation to merge)
  - database:

```
canun:database (develop)$ git switch -c feature/docker-compose
Switched to a new branch 'feature/docker-compose'
```

```
canun:database (feature/docker-compose)$ git add .
canun:database (feature/docker-compose)$ gs
On branch feature/docker-compose
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    Dockerfile
        new file:   Makefile
        new file:   docker-compose.yml

canun:database (feature/docker-compose)$ git commit -m"feat(docker-compose): Create docker_compose.yml"
[feature/docker-compose f9bd173] feat(docker-compose): Create docker_compose.yml
 3 files changed, 32 insertions(+), 2 deletions(-)
 delete mode 100644 Dockerfile
 create mode 100644 Makefile
 create mode 100644 docker-compose.yml
canun:database (feature/docker-compose)$ git status
On branch feature/docker-compose
nothing to commit, working tree clean
canun:database (feature/docker-compose)$ gl
* f9bd173 (HEAD -> feature/docker-compose) feat(docker-compose): Create docker_compose.yml
* 2f34c1a (origin/main, main, develop) Initial version
* 11c47da Initial commit
* 58e198c Initial scaffold
canun:database (feature/docker-compose)$ git switch develop
Switched to branch 'develop'
canun:database (develop)$ git merge feature/docker-compose
Updating 2f34c1a..f9bd173
Fast-forward
 Dockerfile         |  2 --
 Makefile           | 16 ++++++++++++++++
 docker-compose.yml | 16 ++++++++++++++++
 3 files changed, 32 insertions(+), 2 deletions(-)
 delete mode 100644 Dockerfile
 create mode 100644 Makefile
 create mode 100644 docker-compose.yml
canun:database (develop)$ gl
* f9bd173 (HEAD -> develop, feature/docker-compose) feat(docker-compose): Create docker_compose.yml
* 2f34c1a (origin/main, main) Initial version
* 11c47da Initial commit
* 58e198c Initial scaffold
```

- frontend-service:

```
canun:frontend-service (dev)$ git switch -c feature/header
Switched to a new branch 'feature/header'
canun:frontend-service (feature/header)$ gs
On branch feature/header
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/App.js

no changes added to commit (use "git add" and/or "git commit -a")
canun:frontend-service (feature/header)$ git add .
canun:frontend-service (feature/header)$ git commit -m "feat(header): Add header to frontend"
[feature/header 2c0c9e3] feat(header): Add header to frontend
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:frontend-service (feature/header)$ git status
On branch feature/header
nothing to commit, working tree clean
canun:frontend-service (feature/header)$ gl
* 2c0c9e3 (HEAD -> feature/header) feat(header): Add header to frontend
*   f0eb498 (origin/main, main, dev) Merge branch 'dev'
|\
| *   ce60a63 Small changes
| |\
| | * d43d8f6 update
| | * 0ddfaf0 Small editions
| | * f3a9455 Delete docs
| * | c77a51c editions to git-flow.md
| |/
* | 8f5302c Merge pull request #2 from cesarnunezh/dev
|\|
| * 195983c enable Deploy
|/
*   b08386a Merge pull request #1 from cesarnunezh/dev
|\
| * 053b022 Fix tests
| * d3cc6ae New versions with Jenkinsfile
| * 8686ea3 Added dev environment
| * eff4201 Create Docker for frontend-service
| * 08f8390 feat(frontend): Initial version that shows the landing page
|/
* 1549e30 Initial version
* b11cd86 Initial scaffold
canun:frontend-service (feature/header)$ git switch dev
Switched to branch 'dev'
```

```
canun:frontend-service (feature/header)$ git switch dev
Switched to branch 'dev'
canun:frontend-service (dev)$ git merge feature/header
Updating f0eb498..2c0c9e3
Fast-forward
 src/App.js | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:frontend-service (dev)$ gl
* 2c0c9e3 (HEAD -> dev, origin/feature/header, feature/header) feat(header): Add header to frontend
*   f0eb498 (origin/main, main) Merge branch 'dev'
|\
```

- order-service:

```
canun:order-service (dev)$ git switch -c feature/orders-api
Switched to a new branch 'feature/orders-api'
canun:order-service (feature/orders-api)$ gs
On branch feature/orders-api
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/main.py

no changes added to commit (use "git add" and/or "git commit -a")
canun:order-service (feature/orders-api)$ git add .
canun:order-service (feature/orders-api)$ git commit -m"feat(health): add health route"
[feature/orders-api 36f3b6c] feat(health): add health route
 1 file changed, 2 insertions(+), 2 deletions(-)
canun:order-service (feature/orders-api)$ git switch dev
Switched to branch 'dev'
canun:order-service (dev)$ git merge feature/orders-api
Updating 366e41e..36f3b6c
Fast-forward
 src/main.py | 4 ++--
 1 file changed, 2 insertions(+), 2 deletions(-)
```

- product-service:

```
canun:product-service (dev)$ git switch -c feature/products-api
Switched to a new branch 'feature/products-api'
canun:product-service (feature/products-api)$ git add .
canun:product-service (feature/products-api)$ git commit -m"feat(products-api): health"
[feature/products-api a63410e] feat(products-api): health
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:product-service (feature/products-api)$ git switch dev
Switched to branch 'dev'
canun:product-service (dev)$ git merge feature/products-api
Updating 3360b16..a63410e
Fast-forward
 src/main.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

- Complete release workflow (from develop to main)
  - database:

```
canun:database (dev)$ git switch -c release/1.0.0
Switched to a new branch 'release/1.0.0'
canun:database (release/1.0.0)$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
canun:database (main)$ git merge release/1.0.0
Updating 2f34c1a..6906a39
Fast-forward
 Dockerfile        |  2 +-
 Jenkinsfile       | 28 ++++++++++++++--------------
 Makefile          | 24 ++++++++++++++++++++++++
 docs/decisions.md |  0
 docs/git-flow.md  | 46 ----------------------------------------------
 schema/init.sql   | 84 ++++++++++++++++++++++++
```

- Complete hotfix workflow (from main back to main and develop)
  - database:

```
canun:database (main)$ git switch -c hotfix/bug1
Switched to a new branch 'hotfix/bug1'
canun:database (hotfix/bug1)$ git add .
canun:database (hotfix/bug1)$ git commit -m "fix(bug1): fix bug in schema"
[hotfix/bug1 a742021] fix(bug1): fix bug in schema
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:database (hotfix/bug1)$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 8 commits.
  (use "git push" to publish your local commits)
canun:database (main)$ git merge hotfix/bug1
Updating 6906a39..a742021
Fast-forward
 schema/init.sql | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:database (main)$ git switch dev
Switched to branch 'dev'
Your branch is ahead of 'origin/dev' by 1 commit.
  (use "git push" to publish your local commits)
canun:database (dev)$ git merge main
Updating 6906a39..a742021
Fast-forward
 schema/init.sql | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

  - frontend-service:

```
canun:frontend-service (main)$ git switch -c hotfix/bug2
Switched to a new branch 'hotfix/bug2'
canun:frontend-service (hotfix/bug2)$ git add .
canun:frontend-service (hotfix/bug2)$ gc "fix(bug2): fix bug in hero"
[hotfix/bug2 60bd0c4] fix(bug2): fix bug in hero
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:frontend-service (hotfix/bug2)$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
canun:frontend-service (main)$ git merge hotfix/bug2
Updating 2c0c9e3..60bd0c4
Fast-forward
 src/App.js | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:frontend-service (main)$ git switch dev
Switched to branch 'dev'
canun:frontend-service (dev)$ git merge main
Updating 2c0c9e3..60bd0c4
Fast-forward
 src/App.js | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

  - order-service:

```
canun:order-service (main)$ git switch -c hotfix/bug3
Switched to a new branch 'hotfix/bug3'
canun:order-service (hotfix/bug3)$ git add .
canun:order-service (hotfix/bug3)$ git commit -m"fix(bug3): solve some health api issues"
[hotfix/bug3 78abb30] fix(bug3): solve some health api issues
 1 file changed, 2 insertions(+), 2 deletions(-)
canun:order-service (hotfix/bug3)$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)
canun:order-service (main)$ git merge hotfix/bug3
Updating 36f3b6c..78abb30
Fast-forward
 src/main.py | 4 ++--
 1 file changed, 2 insertions(+), 2 deletions(-)
canun:order-service (main)$ git switch dev
Switched to branch 'dev'
canun:order-service (dev)$ git merge main
Updating 36f3b6c..78abb30
Fast-forward
 src/main.py | 4 ++--
 1 file changed, 2 insertions(+), 2 deletions(-)
```

- product-service:

```
canun:product-service (main)$ git switch -c hotfix/bug4
Switched to a new branch 'hotfix/bug4'
canun:product-service (hotfix/bug4)$ git add .
canun:product-service (hotfix/bug4)$ git commit -m"fix(bug4): health api working"
[hotfix/bug4 518e003] fix(bug4): health api working
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:product-service (hotfix/bug4)$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)
canun:product-service (main)$ git merge hotfix/bug4
Updating a63410e..518e003
Fast-forward
 src/main.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
canun:product-service (main)$
canun:product-service (main)$ git switch dev
Switched to branch 'dev'
canun:product-service (dev)$ git merge main
Updating a63410e..518e003
Fast-forward
 src/main.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

- How all branch types interact in the Git Flow model

```
flowchart TD

develop --> feature
feature -->|PR merge| develop

develop --> release
release -->|merge| main
release -->|merge back| develop

main --> hotfix
hotfix -->|merge| main
hotfix -->|merge back| develop
```

- Documentation explaining your design decisions and rationale: available here: `docs/git-flow.md`

Phase 2: Containerization with Docker

Deliverables:

- Dockerfiles for all services
  - database: Available here
  - frontend-service: Available here
  - order-service: Available here
  - product-service: Available here
- Docker Compose configuration

```
services:

web-dev:
```

```
    build:
      context: frontend-service
      target: dev
    image: cesarnunezh/frontend-service:dev
    ports:
      - "3000:3000"
    volumes:
      - ./frontend-service:/app
      - /app/node_modules
    env_file: .env
    profiles:
      - dev

  web:
    build:
      context: frontend-service
      target: production
    image: cesarnunezh/frontend-service:prod
    ports:
      - "3000:80"
    env_file: .env
    profiles:
      - prod

  database:
    build:
      context: database
      target: database
    image: cesarnunezh/database-service:latest
    restart: unless-stopped
    env_file: .env
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d database"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
    networks:
      - app-network

  orders-api:
    build:
      context: order-service
      target: runtime
    image: cesarnunezh/orders-api:latest
    ports:
      - "8050:8050"
    env_file:
      - .env
    depends_on:
```

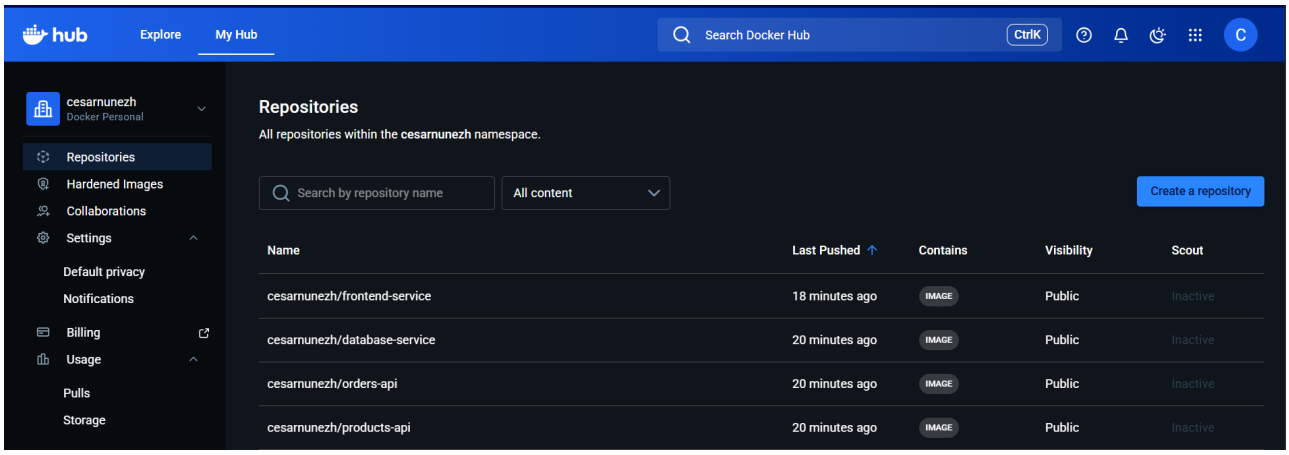
```
      database:
        condition: service_healthy
    networks:
      - app-network

  products-api:
    build:
      context: product-service
      target: runtime
    image: cesarnunezh/products-api:latest
    ports:
      - "8070:8070"
    env_file:
      - .env
    depends_on:
      database:
        condition: service_healthy
    networks:
      - app-network

  networks:
    app-network:

  volumes:
    postgres_data:
```

- Container security scan reports: available here
- Screenshot showing all images successfully pushed to Docker Hub:



- Docker Hub repository URLs for each service:
  - database: https://hub.docker.com/repository/docker/cesarnunezh/database-service/general
  - frontend-service: https://hub.docker.com/repository/docker/cesarnunezh/frontend-service/general
  - order-service: https://hub.docker.com/repository/docker/cesarnunezh/orders-api/general
  - product-service: https://hub.docker.com/repository/docker/cesarnunezh/products-api/general

Phase 3: CI/CD Pipeline with Jenkins

Deliverables:

- Screenshots of Jenkins server with pipelines configured

All +

S	W	Name ↓	Last Success	Last Failure	Last Duration
		database	9 hr 15 min #log	N/A	42 min
		frontend-service	8 hr 40 min #log	N/A	2.2 sec
		order-service	2 hr 58 min #log	N/A	1.5 sec
		product-service	9 hr 19 min #log	N/A	25 min

- database:

database

Branches (3) Pull Requests (0)

S	W	Name ↓	Last Success	Last Failure	Last Duration
		dev	9 hr 5 min #3	N/A	23 min
		feature/jenkins	9 hr 5 min #1	N/A	26 min
		main	3 min 23 sec #3	5 days 9 hr #2	51 sec

- frontend-service:

frontend-service

Branches (4) Pull Requests (0)

S	W	Name ↓	Last Success	Last Failure	Last Duration
		dev	1 min 13 sec #10	11 hr #5	16 sec
		~~feature/docker~~	1 hr 3 min #6	2 hr 44 min #6	26 sec
		main	31 min #13	9 hr 59 min #11	29 min
		~~release/1.0.0~~	9 hr 53 min #1	55 min #2	35 sec

- order-service:

order-service

Branches (4) Pull Requests (0)

S Success	W	Name ↓	Last Success	Last Failure	Last Duration
		dev	7 hr 8 min #9	7 hr 16 min #8	20 sec
		feature/orders-api	7 hr 8 min #8	5 days 18 hr #5	23 sec
		main	7 hr 3 min #4	5 days 7 hr #3	29 sec
		release/1.0.0	7 hr 15 min #1	N/A	29 sec

- product-service:

product-service

Branches (3) Pull Requests (0)

S	W	Name ↓	Last Success	Last Failure	Last Duration
		dev	8 hr 26 min #8	5 days 21 hr #5	14 sec
		feature/product-api	8 hr 27 min #5	9 hr 4 min #4	16 sec
		main	1 min 6 sec #4	5 days 9 hr #3	1 min 0 sec

- Jenkinsfiles for all services:
  - Shared folder: Available here
  - database: Available here
  - frontend-service: Available here
  - order-service: Available here
  - product-service: Available here
- Screenshots of pipeline executions for each environment (Build, Dev, Staging, Prod):

- Build/Dev:

- Staging:

- Prod:

Phase 4: Infrastructure as Code with Terraform

Deliverables:

- Terraform modules and configurations: Available here
- State management setup: Available here
- Screenshots showing:
  - Terraform plan output for each environment
    - Dev: Available here
    - Staging: Available here
    - Prod: Available here
  - Successful terraform apply results
    - Dev: Available here
    - Staging: Available here
    - Prod: Available here
  - Running Minikube cluster and local resources

```
canun:DevOpsProject (main)$ minikube profile list
```

PROFILE	DRIVER	RUNTIME	IP	VERSION	STATUS	NODES	ACTIVE PROFILE	ACTIVE KUBECONTEXT
devops-dev-minikube	docker	docker	192.168.58.2	v1.32.0	OK	1		
devops-prod-minikube	docker	docker	192.168.67.2	v1.32.0	OK	1		
devops-staging-minikube	docker	docker	192.168.76.2	v1.32.0	OK	1		*
minikube	docker	docker	192.168.49.2	v1.35.0	Stopped	1	*	

  - Terraform outputs that will be used by Jenkins
    - Dev:

```
docker_network_name = "devops-dev-network"
docker_registry_namespace = "cesarnunezh"
environment = "dev"
image_repository_prefix = "cesarnunezh"
jenkins_port = 8080
jenkins_url = "http://localhost:8080"
kubernetes_namespace = "dev"
minikube_cluster_ip = ""
minikube_kubeconfig_context = "devops-dev-minikube"
minikube_kubeconfig_path = "/home/canun/.kube/config"
minikube_profile = "devops-dev-minikube"
name_prefix = "devops-dev"
```

```
postgres_connection_string = <sensitive>
postgres_db = "database"
postgres_host = "localhost"
postgres_port = 5432
service_base_urls = {
  "frontend" = "http://localhost:3000"
  "jenkins" = "http://localhost:8080"
  "orders" = "http://localhost:8050"
  "products" = "http://localhost:8070"
}
```

- Staging:

```
docker_network_name = "devops-staging-network"
docker_registry_namespace = "cesarnunezh"
environment = "staging"
image_repository_prefix = "cesarnunezh"
jenkins_port = 8081
jenkins_url = "http://localhost:8081"
kubernetes_namespace = "staging"
minikube_cluster_ip = ""
minikube_kubeconfig_context = "devops-staging-minikube"
minikube_kubeconfig_path = "/home/canun/.kube/config"
minikube_profile = "devops-staging-minikube"
name_prefix = "devops-staging"
postgres_connection_string = <sensitive>
postgres_db = "database"
postgres_host = "localhost"
postgres_port = 5433
service_base_urls = {
  "frontend" = "http://localhost:3100"
  "jenkins" = "http://localhost:8081"
  "orders" = "http://localhost:8150"
  "products" = "http://localhost:8170"
}
```

- Prod:

```
docker_network_name = "devops-prod-network"
docker_registry_namespace = "cesarnunezh"
environment = "prod"
image_repository_prefix = "cesarnunezh"
jenkins_port = 8082
jenkins_url = "http://localhost:8082"
kubernetes_namespace = "prod"
minikube_cluster_ip = ""
minikube_kubeconfig_context = "devops-prod-minikube"
minikube_kubeconfig_path = "/home/canun/.kube/config"
minikube_profile = "devops-prod-minikube"
```

```
  name_prefix = "devops-prod"
  postgres_connection_string = <sensitive>
  postgres_db = "database"
  postgres_host = "localhost"
  postgres_port = 5434
  service_base_urls = {
    "frontend" = "http://localhost:3200"
    "jenkins" = "http://localhost:8082"
    "orders" = "http://localhost:8250"
    "products" = "http://localhost:8270"
  }
```

Phase 5: Kubernetes Deployment

Deliverables:

- Kubernetes manifests for all services: Available here
- Screenshots showing:
  - Running pods in different namespaces:

```
canun:DevOpsProject (main)$ kubectl get pods -A
NAMESPACE        NAME                                        READY   STATUS      RESTARTS      AGE
devops-dev       database-service-86bd6c466f-mhlh6           1/1     Running     0             25m
devops-dev       products-api-green-75dd46df59-nls9z         1/1     Running     0             2m9s
devops-prod      database-service-86bd6c466f-79wjw           1/1     Running     0             25m
devops-prod      products-api-green-75dd46df59-hjxqz         1/1     Running     0             23s
devops-prod      products-api-green-75dd46df59-kv46n         1/1     Running     0             23s
devops-staging   database-service-86bd6c466f-s2ffl           1/1     Running     0             25m
devops-staging   frontend-service-green-d44c76b84-gnw2c      1/1     Running     0             3m2s
```

  - Services and ingress configuration:

```
canun:DevOpsProject (main)$ kubectl get svc,ingress -n devops-dev
NAME                                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/database-service              ClusterIP   10.98.193.225    <none>        5432/TCP         33m
service/frontend-service              ClusterIP   10.106.63.87     <none>        80/TCP           33m
service/frontend-service-nodeport     NodePort    10.108.18.48     <none>        80:30080/TCP     33m
service/orders-api                    ClusterIP   10.97.190.123    <none>        8050/TCP         33m
service/orders-api-nodeport           NodePort    10.106.217.201   <none>        8050:30050/TCP   33m
service/products-api                  ClusterIP   10.96.140.7      <none>        8070/TCP         33m
service/products-api-nodeport         NodePort    10.108.154.244   <none>        8070:30070/TCP   33m

NAME                                        CLASS   HOSTS              ADDRESS        PORTS   AGE
ingress.networking.k8s.io/devops-ingress    nginx   dev.devops.local   192.168.76.2   80      33m
canun:DevOpsProject (main)$ kubectl describe ingress devops-ingress -n devops-dev
Name:             devops-ingress
Labels:           <none>
Namespace:        devops-dev
Address:          192.168.76.2
Ingress Class:    nginx
Default backend:  <default>
Rules:
  Host              Path  Backends
  ----              ----  --------
  dev.devops.local
                    /products(/|$)(.*)   products-api:8070 (10.244.0.41:8070)
                    /orders(/|$)(.*)     orders-api:8050 (10.244.0.42:8050)
                    /()(.*)              frontend-service:80 (10.244.0.40:80)
Annotations:        nginx.ingress.kubernetes.io/rewrite-target: /$2
Events:
  Type    Reason  Age                From                      Message
  ----    ------  ----               ----                      -------
  Normal  Sync    33m (x2 over 33m)  nginx-ingress-controller  Scheduled for sync
```

  - Successful deployment rollout

- Before:

```
canun:DevOpsProject (main)$   kubectl get deploy -n devops-dev
  kubectl get svc -n devops-dev -o jsonpath='{range .items[*]}{.metadata.name}{" => "}{.spec.selector.version}{"\n"}{end}'
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
database-service        1/1     1            1           45m
frontend-service-blue   0/0     0            0           45m
frontend-service-green  1/1     1            1           45m
orders-api-blue         0/0     0            0           45m
orders-api-green        1/1     1            1           45m
products-api-blue       0/0     0            0           45m
products-api-green      1/1     1            1           45m
database-service =>
frontend-service => green
frontend-service-nodeport => green
orders-api => green
orders-api-nodeport => green
products-api => green
products-api-nodeport => green
```

- After:

```
canun:DevOpsProject (main)$   export IMAGE_URI=cesarnunezh/frontend-service:latest
  export MUTABLE_TAG=latest
  bash scripts/k8s-deploy.sh dev
Waiting for deployment "frontend-service-blue" rollout to finish: 0 of 1 updated replicas are available...
deployment "frontend-service-blue" successfully rolled out
canun:DevOpsProject (main)$ kubectl rollout status deployment/frontend-service-blue -n devops-dev
deployment "frontend-service-blue" successfully rolled out
canun:DevOpsProject (main)$ kubectl get deploy -n devops-dev
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
database-service        1/1     1            1           46m
frontend-service-blue   1/1     1            1           46m
frontend-service-green  0/0     0            0           46m
orders-api-blue         0/0     0            0           46m
orders-api-green        1/1     1            1           46m
products-api-blue       0/0     0            0           46m
products-api-green      1/1     1            1           46m
canun:DevOpsProject (main)$ kubectl get svc -n devops-dev -o jsonpath='{range .items[*]}{.metadata.name}{" => "}{.spec.selector.version}{"\n"}{end}'
database-service =>
frontend-service => blue
frontend-service-nodeport => blue
orders-api => green
orders-api-nodeport => green
products-api => green
products-api-nodeport => green
canun:DevOpsProject (main)$   export IMAGE_URI=cesarnunezh/products-api:latest
  export MUTABLE_TAG=latest
  bash scripts/k8s-deploy.sh dev
  kubectl rollout status deployment/products-api-blue -n devops-dev
Waiting for deployment "products-api-blue" rollout to finish: 0 of 1 updated replicas are available...
deployment "products-api-blue" successfully rolled out
deployment "products-api-blue" successfully rolled out
canun:DevOpsProject (main)$   export IMAGE_URI=cesarnunezh/orders-api:latest
  export MUTABLE_TAG=latest
  bash scripts/k8s-deploy.sh dev
  kubectl rollout status deployment/orders-api-blue -n devops-dev
Waiting for deployment "orders-api-blue" rollout to finish: 0 of 1 updated replicas are available...
deployment "orders-api-blue" successfully rolled out
deployment "orders-api-blue" successfully rolled out
canun:DevOpsProject (main)$ kubectl get deploy -n devops-dev
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
database-service        1/1     1            1           48m
frontend-service-blue   1/1     1            1           48m
frontend-service-green  0/0     0            0           48m
orders-api-blue         1/1     1            1           48m
orders-api-green        0/0     0            0           48m
products-api-blue       1/1     1            1           48m
products-api-green      0/0     0            0           48m
```

- Documentation explaining your deployment strategy (rolling updates, blue-green, or canary)

Phase 6: Integration Validation

Objective: Demonstrate that all components work together as a complete DevOps pipeline.

Deliverables:

- Screenshots or screen recording showing the complete flow from code commit to production
- Evidence of successful deployments in all environments:
  - kubectl output showing running pods
  - Application accessible via browser/API
  - Correct image tags in each environment
- Brief report summarizing any challenges faced and how they were resolved